Layered Architecture of Application

Dr Harshad Prajapati
13 Jan 2024

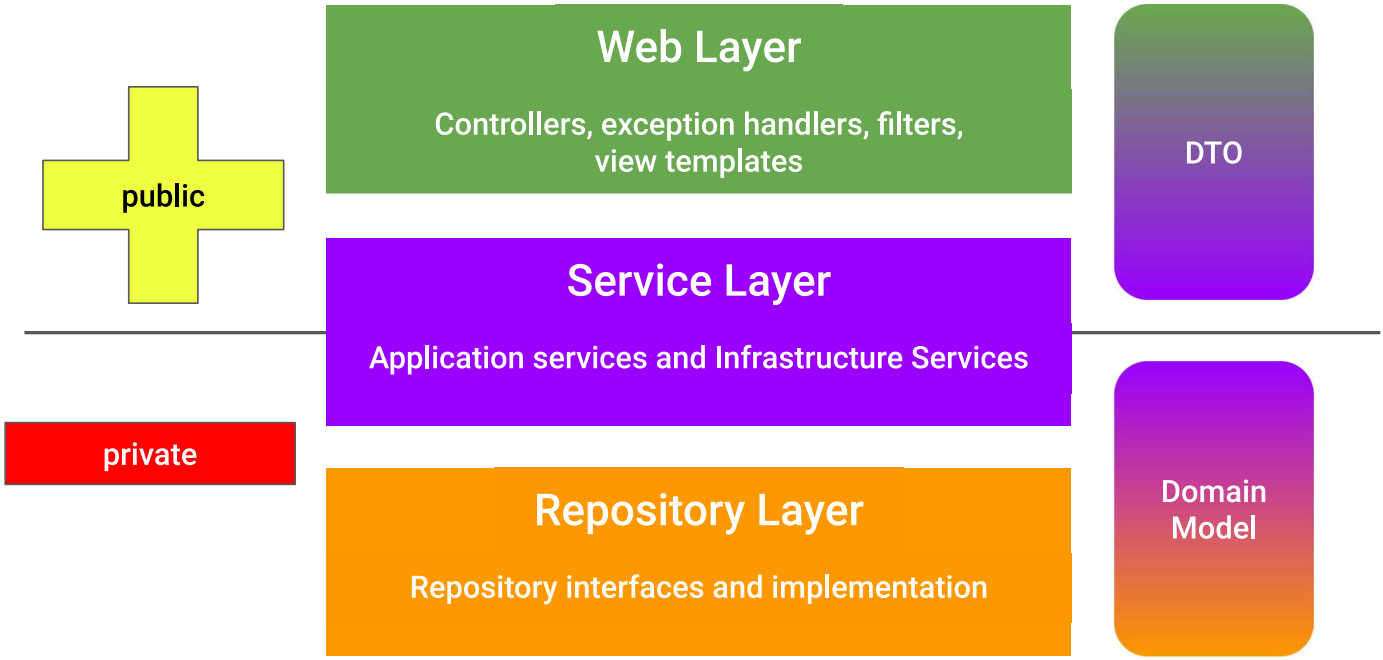
Responsibilities of a Web application

- Web app (REST API) needs to process the user's input (API request) and return the correct response back to the user.
- It needs an exception handling mechanism that provides reasonable error messages to the user.
- It needs a transaction management strategy.
- It needs to handle both authentication and authorization.
- It needs to implement the business logic of the application.
- It needs to communicate with the used data storage and other external resources.

Three Layer Architecture

- We can fulfill all the concerns using only three layers:
  - The web layer,
  - The service layer,
  - The repository layer.

Three Layer Architecture



Web Layer

- Process user's input or API request and sends response.
- It also handles exceptions.
- The web layer should handle only data transfer objects.

Service Layer

- The service layer takes data transfer objects (and basic types) as method parameters.
- It can handle domain model objects but it can return only data transfer objects to the web layer.

Service Layer

- Service layer works as transaction boundary.
- This layer contains
  - Application Services, and
  - Infrastructure Services,
- Application Service:
  - Provides public API of the service layer.
  - Act as a transaction boundary and are responsible for authorization.
- Infrastructure Service:
  - Contains code that communicates with external resources such as the file systems, databases, or email servers.
  - These methods are generally used by more than one application service.

The repository layer

- Responsible for communicating with the used data storage.
- It takes entities (and basic types) as method parameters and returns entities (and basic types) and collection of entities.

Data Transfer Object - DTO

- DTO object works as a simple data container.
- It is used to carry data between different processes and between the layers of an application.
- It does not contain any business logic.
- It can have only data members, but generally kept private and exposed via public methods.

Domain Model

- Domain model does not bind with view.
- It can contain the following objects:
  - Entity,
  - Value Object,
  - Domain service.

Domain Model

- Entity:
  - It is defined by its identity which stays unchanged through its entire lifecycle.
- Value Object (It is like an enum)
  - Describes a property or a thing.
    (e.g. LoanApplication.RECEIVED, LoanApplication.PENDING, etc.)
  - These objects do not have their own identity or lifecycle. They are immutable.
  - Two VOs are equal if their values are equal. (e.g., Integer)
  - The lifecycle of a value object is bound to the lifecycle of an entity.

Domain Model

- Domain service:
  - It is a stateless class.
  - Provides operations that are not naturally part of an entity or a value object.

Code Structure

- There are two possible code structures:
  - Java packages by Layer.
  - Java packages by Feature.

Packages by Layer

- We create a package for each layer:
  - dto
  - controller
  - service
  - entity / persistence
  - dao / repository
- A particular package contains layer specific classes of all functionalities.
  - For example, a controller package contains all controllers of whole application.
    - SearchController, LoginController, OrderController

Packages by Feature

- We create a package for each feature:
  - login
  - search
  - order
- A particular package contains all classes of that particular functionality.
  - For example, a search package contains all classes required to build search functionality.
    - SearchDto, SearchController, SearchService, Search (entity), SearchRepository.

Important Annotations

- @Component
- @Autowired
- @Bean
- @Controller
- @RestController
- @Repository
- @Service

Important Annotations

- @Controller
  - It is general purpose annotation used to create web applications.
  - Methods within a @Controller-annotated class return the name of a view to be rendered by the view resolver.
- @RestController
  - It is a specialized version of @Controller that combines @Controller and @ResponseBody. used in creating REST or Web Services.
  - Methods in a class marked with @RestController are assumed to return data (rather than view name) that should be serialized into the HTTP response body.

Important Annotations

- @Repository
  - Its job is to catch persistence-specific exceptions and rethrow those exceptions as one of Spring's unified unchecked exceptions.
- @Service
  - They are used to indicate that the class is holding business logic.
  - Used in Service Layer.

Important Annotations

- @Autowired
  - It enables automatic dependency injection of the following objects
    - @Component,
    - @Configuration,
    - @Bean returned object,
  - Autowired works on
    - Field or property,
    - Setter method,
    - Constructor.
  - We can use @Qualifier to select object if we have multiple objects that can be injected.

Important Annotations

- @Component
  - It is used to mark a bean as Spring's managed component.
- @Bean
  - This annotation is used on method, and not on the class as done by @Component, to indicate that the method returns a bean that is managed by Spring.

Need of Service Layer

- Service layer is an **intermediate layer** for **custom business logic**.
- In service layer, we can **integrate data** from **multiple sources** (DAO or repositories)

References

- https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/
-